Software Requirements Specification

CampusGPT

An AI-Powered University Intelligence and Student Assistance Platform

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

Computer Science and Engineering

Specialization: **Cyber Security**

Prepared by

Name	SAP ID	Batch
Ankur Karn	500118986	11
Ashmita Sharma	500122422	11
Ananya Singh	500123837	11
Ayushi Mishra	500121986	11

Under the guidance of

Mr. Yogesh Rathia

Project Mentor

School of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES

Dehradun – 248007, Uttarakhand

October 2026 | Version 1.1

Contents

Revision History **iv**

Document Conventions **v**

1 SRS Introduction **1**

1.1 Purpose of the Project 1

1.2 Target Beneficiary 1

1.3 Functional Requirements 2

2 SRS Project Description **4**

2.1 Reference Algorithms and Concepts 4

2.2 SWOT Analysis 6

2.3 Project Schedule, PERT and Gantt Analysis 6

3 SRS Non-Functional Requirements **8**

3.1 Performance Requirements 8

3.2 Security Requirements 8

3.3 Software Quality Attributes (ISO/IEC 25010) 8

A Glossary **10**

B Analysis Model **11**

C Issues List **13**

References **14**

List of Figures

2.1 Gantt chart of the CampusGPT schedule (13 Oct – 12 Nov 2026). 7

B.1 Level-0 Context Diagram of CampusGPT. 11
B.2 System architecture of CampusGPT. 12
B.3 Level-1 Data Flow Diagram of CampusGPT. 12

List of Tables

1.1 Target beneficiaries and the benefit to each 1
1.2 Functional requirements of CampusGPT 2

2.1 Student questions and the method used to answer each 4
2.2 SWOT analysis of CampusGPT 6
2.3 PERT activity table (durations in days) 7

3.1 Performance requirements of CampusGPT 8
3.2 Security requirements of CampusGPT 8
3.3 Software quality attributes mapped to ISO/IEC 25010 9

A.1 Glossary of terms and abbreviations 10

C.1 Living register of engineering issues and roadmap resolutions 13

Revision History

Date	Change	Reason for Changes
Tue, 13 Oct 2026	Version 1.0. First complete draft: introduction, beneficiaries, functional requirements, reference algorithms, SWOT analysis, PERT and Gantt schedule, non-functional and security requirements, glossary, context diagram, and issues list.	First submission for mentor review.
Tue, 13 Oct 2026	Version 1.1. Added system architecture diagram and Level-1 data flow diagram (Appendix B); document ingestion now runs in a separate worker process (FR-14, Algorithm 1).	Alignment with the Technical Design Document for evaluation.

Document Conventions

- The document follows the structure recommended by IEEE Std 830-1998 [1].
- Requirements are written with **"shall"**. Each has an ID such as **FR-06** (functional), **NFR-P1** (non-functional, performance) or **NFR-S1** (non-functional, security) so it can be traced directly to design and testing.
- Priorities are **High** (needed for release), **Medium** (planned for release), and **Low** (good to have).
- Performance figures in Chapter 3 are **targets** to be verified during testing, not measured results.
- All student records and policy documents referred to in this document are **synthetic** (fictional) and are labelled as such inside the system.

CHAPTER 1

SRS Introduction

1.1 Purpose of the Project

CampusGPT is an AI-powered helpdesk platform that gives university students a single conversational interface to their academic and administrative information. Instead of searching several portals, PDFs and notice boards, a student asks a question in natural language, such as "How many classes must I attend to reach 75% in DBMS?". The system answers by combining two sources: the student's own records, fetched from a PostgreSQL database through predefined, permission-checked backend tools, and university policy documents, searched with Retrieval-Augmented Generation (RAG) [2] and cited in the answer. The language model never queries the database directly, numerical results are computed by deterministic code, and access to student data is enforced by the backend rather than by prompt instructions. Because the project has no access to real university systems, all data is synthetic, and the design permits a later switch to authorized university APIs.

1.2 Target Beneficiary

Table 1.1 delineates target user groups and direct benefits.

Table 1.1: Target beneficiaries and the benefit to each

Beneficiary	Who they are	How CampusGPT helps
Students	Undergraduate students enrolled in university programs	Instant answers about their attendance, exams, assignments, fees and the rules that apply, with sources shown; a prioritised weekly summary.
University Administrators	Academic office and helpdesk staff	Fewer repetitive queries; policy documents and notices published once and immediately answerable in chat.
Faculty Advisors	Teachers mentoring student groups	Students arrive with accurate, policy-backed information (direct faculty access is planned for a later version).
University IT Department	Teams maintaining ERP and portal systems	A conversational layer that connects to existing systems through a defined adapter, without replacing them.

1.3 Functional Requirements

The functional requirements of CampusGPT are cataloged in Table 1.2.

Table 1.2: Functional requirements of CampusGPT

ID	Requirement Name	Priority	The system shall...
FR-01	Synthetic Data Generation	High	Generate a reproducible (seeded) synthetic university of 3,000–5,000 students with consistent programs, courses, enrollments, attendance, exams, assignments and fees.
FR-02	Synthetic Labelling	High	Label every generated record and policy document as synthetic demonstration data.
FR-03	Authentication	High	Authenticate users by email and password, storing passwords as Argon2id hashes [3].
FR-04	Role-Based Access	High	Support *student* and *administrator* roles and restrict every endpoint by role [4].
FR-05	Student Data Isolation	High	Derive the student identity for every data request from the authenticated session, never from user or model input.
FR-06	Student Dashboard	High	Display the logged-in student's attendance, upcoming exams, pending assignments and fee status.
FR-07	Conversational Chat	High	Accept natural-language questions and keep per-user conversation history.
FR-08	Tool Calling	High	Answer student-specific questions only through predefined, read-only backend tools; the model shall not generate or execute SQL.
FR-09	Attendance Calculator	High	Compute classes required to reach, or classes that may be missed while staying above, an attendance threshold using deterministic code.
FR-10	Document Ingestion	High	Extract, clean, chunk and embed policy documents (PDF or text) and store the vectors in PostgreSQL with pgvector.
FR-11	Hybrid Retrieval	Medium	Retrieve relevant chunks by combining vector similarity and keyword search using Reciprocal Rank Fusion [5].
FR-12	Source Citations	High	Cite, in every policy-based answer, the document and section of each chunk actually used, and mark database-derived facts as such.
FR-13	Weekly Summary	High	Produce a prioritised list of items needing the student's attention in the next seven days.

ID	Requirement Name	Priority	The system shall...
FR-14	Admin Document Upload	Medium	Allow administrators to upload, replace and delete policy documents and notices, re-indexing them automatically in a separate worker process.
FR-15	Audit Logging	Medium	Record logins, tool executions and administrative actions with user, time and outcome, excluding sensitive field values.
FR-16	AI Outage Fallback	Medium	Keep the dashboard and all non-chat pages working, and show a clear message in chat, when the language-model service is unavailable.
FR-17	Suggested Questions	Low	Offer example questions to new users in the chat interface.

CHAPTER 2

SRS Project Description

2.1 Reference Algorithms and Concepts

CampusGPT maps student questions to verified data sources and methods (Table 2.1). The orchestration follows the tool-use pattern of ReAct [6] and Toolformer [7], restricted to a fixed tool catalogue.

Table 2.1: Student questions and the method used to answer each

Student Question	**Computational Method**	**Used in**
What is my attendance in DBMS?	Session-scoped tool `get_my_attendance` (SQL inside the backend)	Chat, Dashboard
How many classes must I attend to reach 75%?	Deterministic formula (Algorithm 3)	Attendance calculator tool, Chat
What is the attendance rule for exams?	Hybrid RAG: pgvector HNSW [8] search + full-text search, fused by RRF	Chat (with citations)
Can I sit for my exam?	Tool result + retrieved policy, combined by the LLM	Chat
What do I need to take care of this week?	Rule-based urgency scoring, explained by the LLM	Weekly Summary

Algorithm 1: Policy Document Ingestion and Indexing

1. Validate the upload (administrator role, PDF/TXT type, size $\leq 10\,\text{MB}$); queue an ingestion job and notify the worker process, which performs steps 2–6.
2. Extract text page by page; remove headers, footers and repeated whitespace.
3. Split into chunks of about 500 tokens with 50-token overlap, keeping the document title, section heading and page number as metadata.
4. Embed each chunk with a local open-source embedding model (BGE-small [9], 384 dimensions) on the CPU.
5. Store chunks, metadata and vectors in PostgreSQL; build the HNSW vector index and the full-text index.
6. Mark the previous version of the same document as superseded so that only the current version is retrieved.

Algorithm 2: Authenticated Question Answering with Tools and Citations

1. Authenticate the request from the session cookie; load `user_id` and role.
2. Send the question, conversation history and the tool list permitted for the role to the LLM through the provider adapter.
3. For each tool call requested by the LLM: check the tool is in the role's allow-list, validate its arguments against the tool's schema, inject `student_id` from the session, execute it, and write an audit-log entry.
4. Return tool results and retrieved policy chunks (each with a chunk ID) to the LLM, wrapped as untrusted data [10].
5. Receive the final answer; verify that every cited chunk ID was actually retrieved and drop any citation that was not.
6. Return the answer with its database and document sources; store the turn in the conversation history.

Algorithm 3: Attendance Requirement Calculation

1. Read attended classes a and total classes held t for the course; read threshold p (e.g. 0.75) from configuration.
2. If $a/t < p$: classes needed $x = \left\lceil \dfrac{p\,t - a}{1 - p} \right\rceil$, the smallest x with $(a + x)/(t + x) \geq p$.
3. Else: classes that may be missed $m = \left\lfloor \dfrac{a - p\,t}{p} \right\rfloor$, the largest m with $a/(t + m) \geq p$.
4. Return a, t, current percentage, and x or m, labelled as a calculated value.

2.2 SWOT Analysis

Table 2.2 presents the SWOT analysis of the CampusGPT platform.

Table 2.2: SWOT analysis of CampusGPT

STRENGTHS (Internal)	**WEAKNESSES (Internal)**
• Answers grounded in database records and cited policy text. • Student data isolation enforced in the backend, not by prompts. • Deterministic calculations for attendance and deadlines. • Fully free stack: open-source software, local embeddings, free-tier LLM API. • Swappable LLM provider through a single adapter.	• Uses synthetic data only; no live university integration. • Free-tier LLM APIs impose rate limits. • English only in the first version. • Scanned (image-only) PDFs are not supported. • Smaller models may select the wrong tool for complex questions.
OPPORTUNITIES (External)	**THREATS (External)**
• Integration with university ERP systems through authorized APIs. • Multilingual support through Indian-language speech and translation models. • Extension to faculty and administrative staff. • Delivery through mobile apps or messaging platforms.	• Changes in free-tier limits or terms of LLM providers. • New prompt-injection techniques [11]. • Compliance obligations under the DPDP Act, 2023 [12] for real deployments. • Loss of student trust after an incorrect answer [13].

2.3 Project Schedule, PERT and Gantt Analysis

The engineering lifecycle is executed over 31 days (13 October to 12 November 2026), developed iteratively [14]. Table 2.3 documents the activity network, with durations in days. Activities with zero slack form the Critical Path: **A** $\rightarrow$ **B** $\rightarrow$ **D** $\rightarrow$ **E** $\rightarrow$ **G** $\rightarrow$ **H** $\rightarrow$ **J** $\rightarrow$ **M** $\rightarrow$ **N** (Total: 31 days). Figure 2.1 shows the corresponding Gantt chart.

Table 2.3: PERT activity table (durations in days)

ID	Activity Description	Dur.	Pred.	ES	EF	LS	Slack
A	Problem study & requirements (SRS)	2	–	0	2	0	**0**
B	Literature review & toolchain setup	2	A	2	4	2	**0**
C	Synthetic policy documents	5	A	2	7	11	9
D	Database schema & migrations	3	B	4	7	4	**0**
E	Synthetic data generator	4	D	7	11	7	**0**
F	Authentication & RBAC	3	D	7	10	8	1
G	Student APIs & dashboard	5	E, F	11	16	11	**0**
H	LLM adapter & tool calling	5	G	16	21	16	**0**
I	RAG ingestion, retrieval & citations	5	C, D	7	12	16	9
J	Weekly summary (Campus Intelligence)	3	H, I	21	24	21	**0**
K	Admin document upload	2	I	12	14	22	10
L	Evaluation benchmark questions	5	C	7	12	19	12
M	Security testing & evaluation	4	J, K, L	24	28	24	**0**
N	Report, demonstration & defense	3	M	28	31	28	**0**

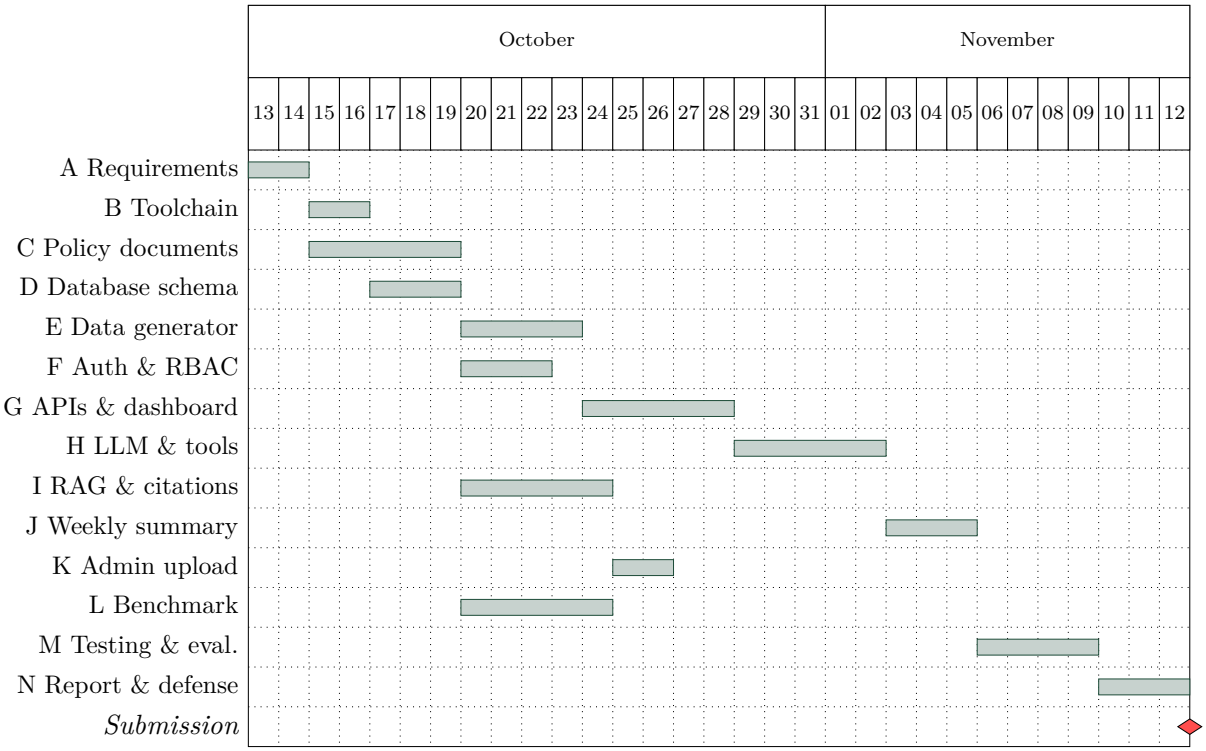


Figure 2.1: Gantt chart of the CampusGPT schedule (13 Oct – 12 Nov 2026).

CHAPTER 3

SRS Non-Functional Requirements

3.1 Performance Requirements

Table 3.1 itemizes the performance targets of CampusGPT. They are measured on the development machine (Intel Core i5-12450HX, 8 GB RAM) with the full synthetic dataset loaded.

Table 3.1: Performance requirements of CampusGPT

ID	Requirement Statement	Measurable Target
NFR-P1	Dashboard API response time	$<$ 300 ms (95th percentile)
NFR-P2	Single tool execution (database query)	$<$ 100 ms (95th percentile)
NFR-P3	Hybrid document retrieval	$<$ 500 ms per query
NFR-P4	End-to-end chat answer, hosted LLM	$<$ 8 s median
NFR-P5	Synthetic data generation, 5,000 students	$<$ 5 min
NFR-P6	Ingestion of a 20-page policy PDF	$<$ 60 s on CPU
NFR-P7	Backend memory incl. embedding model	$<$ 1.5 GB resident
NFR-P8	Automated test suite execution	$<$ 2 min

3.2 Security Requirements

Table 3.2 lists the security requirements, derived from the OWASP Top 10 [15] and the OWASP Top 10 for LLM Applications [11].

Table 3.2: Security requirements of CampusGPT

ID	The system shall...
NFR-S1	Return zero records belonging to another student in all cross-user access tests, whether attempted through the API, by changing IDs in the browser, or through prompts.
NFR-S2	Reject any tool call outside the caller's role allow-list, regardless of the LLM's request.
NFR-S3	Treat uploaded document text as untrusted data and never as instructions.
NFR-S4	Store secrets only in environment variables, never in the source repository.
NFR-S5	Rate-limit login and chat endpoints per user and per IP address.
NFR-S6	Exclude passwords, tokens and personal fields from application logs.

3.3 Software Quality Attributes (ISO/IEC 25010)

Table 3.3 maps system quality characteristics to the ISO/IEC 25010 standard [16].

Table 3.3: Software quality attributes mapped to ISO/IEC 25010

Quality Attribute	**How CampusGPT Achieves It**
Functional Suitability	Answers student questions from database records and cited policy text; correctness measured on a benchmark question set [17].
Performance Efficiency	Indexed queries, HNSW vector index, local CPU embeddings, and bounded context size.
Compatibility	REST API documented with OpenAPI [18]; LLM provider and university data source isolated behind adapters.
Interaction Capability	Single chat interface with suggested questions, readable formatting and visible sources; responsive dashboard [19].
Reliability	Non-chat features keep working during LLM outages; clear error messages on tool failure.
Security	Session authentication, role-based access, server-side data isolation, audit logs, and input validation.
Maintainability	Modular separation of API, tools, retrieval and provider adapter, with an automated test suite.
Flexibility	Runs on Linux, macOS and Windows; PostgreSQL [20] and pgvector [21] run locally or on a hosted service.

APPENDIX A

Glossary

Table A.1: Glossary of terms and abbreviations

Term	Definition
LLM	Large Language Model: a neural network that generates text from a prompt.
RAG	Retrieval-Augmented Generation: supplying retrieved documents to an LLM so its answer is based on them.
Embedding	A fixed-length numeric vector representing the meaning of a piece of text.
Chunk	A short section of a document that is embedded and retrieved as one unit.
pgvector	A PostgreSQL extension for storing vectors and running similarity search.
HNSW	Hierarchical Navigable Small World: a graph-based index for fast approximate nearest-neighbour search.
Hybrid Search	Retrieval combining vector similarity with keyword (full-text) matching.
RRF	Reciprocal Rank Fusion: a method for merging several ranked result lists into one.
Tool Calling	An LLM requesting that the application run a named function with given arguments.
Prompt Injection	Text crafted to make an LLM ignore its instructions or misuse its tools.
Hallucination	LLM output that is fluent but not supported by the provided sources.
RBAC	Role-Based Access Control: permissions granted according to a user's role.
Argon2id	A memory-hard password-hashing algorithm.
Synthetic Data	Artificially generated records that resemble real data but describe no real person.
DPDP Act	India's Digital Personal Data Protection Act, 2023.

APPENDIX B

Analysis Model

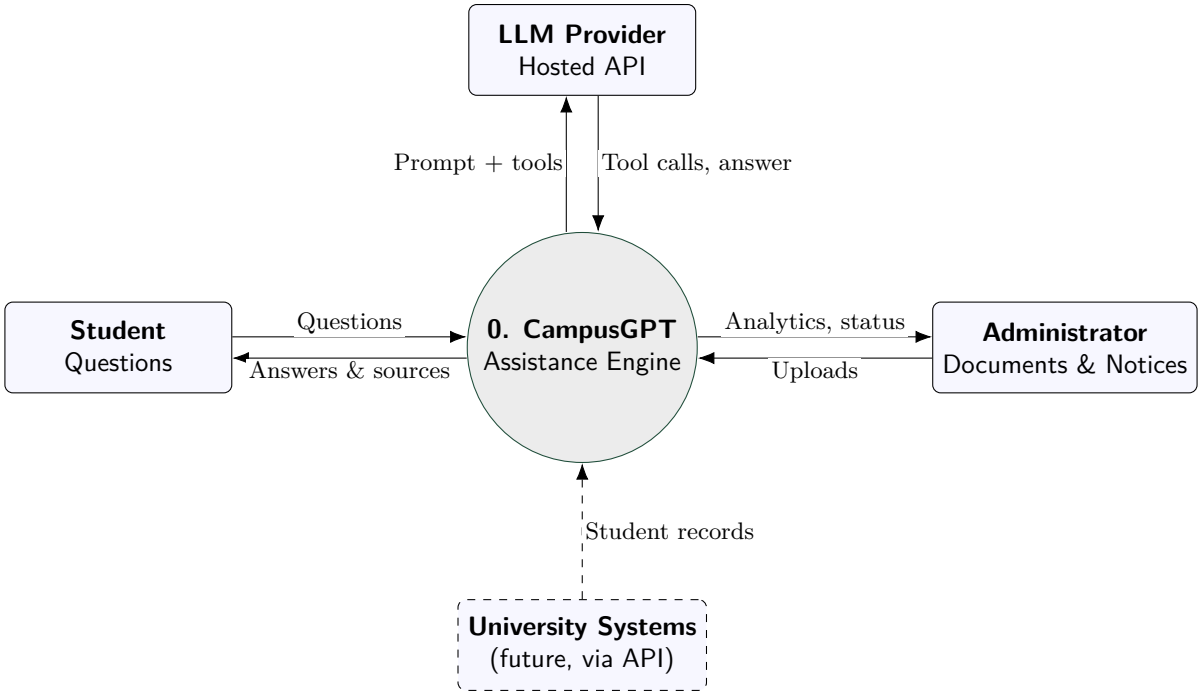


Figure B.1: Level-0 Context Diagram of CampusGPT.

Figure B.1 shows CampusGPT as a single process. Students send questions and receive answers with sources; administrators upload documents and notices; the hosted LLM receives only the prompt, permitted tool definitions and retrieved context. In the current version, student records come from the synthetic PostgreSQL database; the dashed University Systems entity marks the planned integration point.

Figure B.2 shows the system architecture: a browser client, a FastAPI application process, a separate ingestion worker process, and PostgreSQL with pgvector. Figure B.3 decomposes the context diagram into five processes and five data stores. Both diagrams are explained in detail in the Technical Design Document.

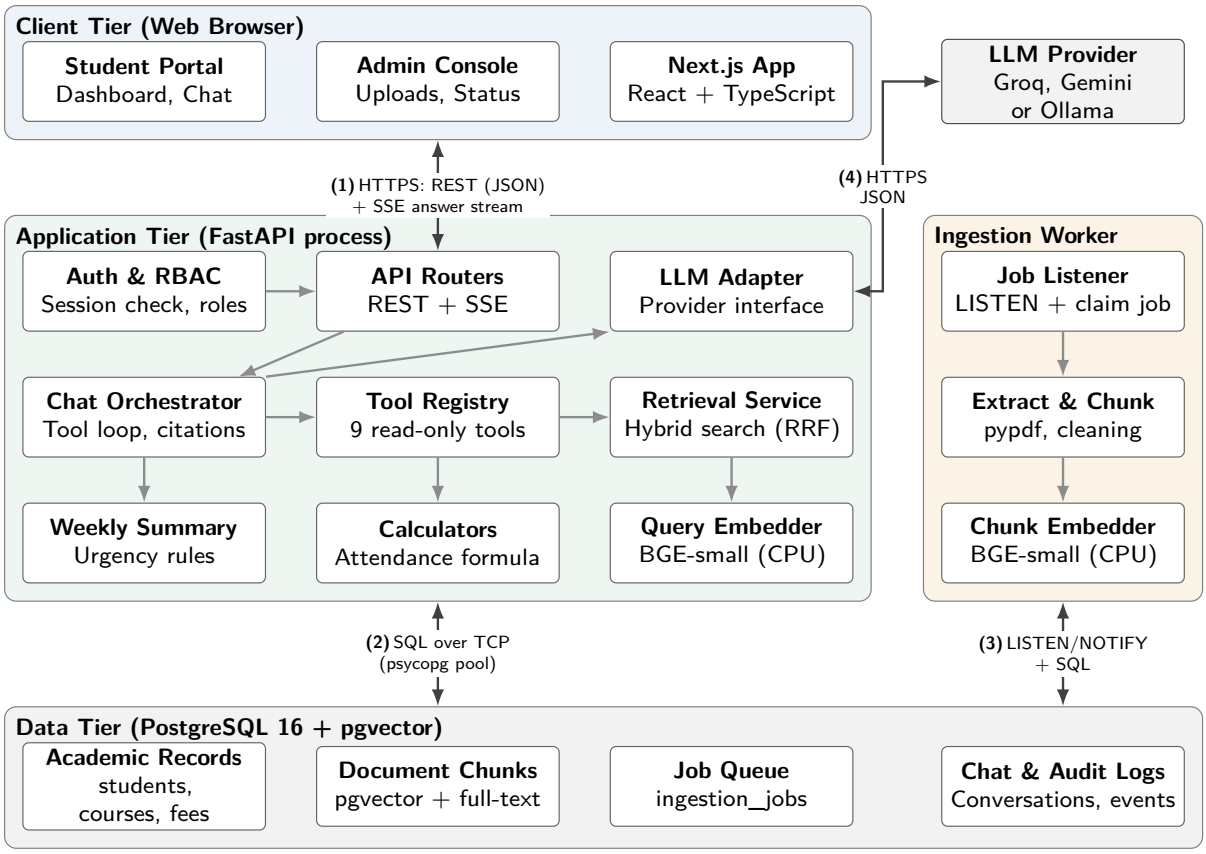


Figure B.2: System architecture of CampusGPT.

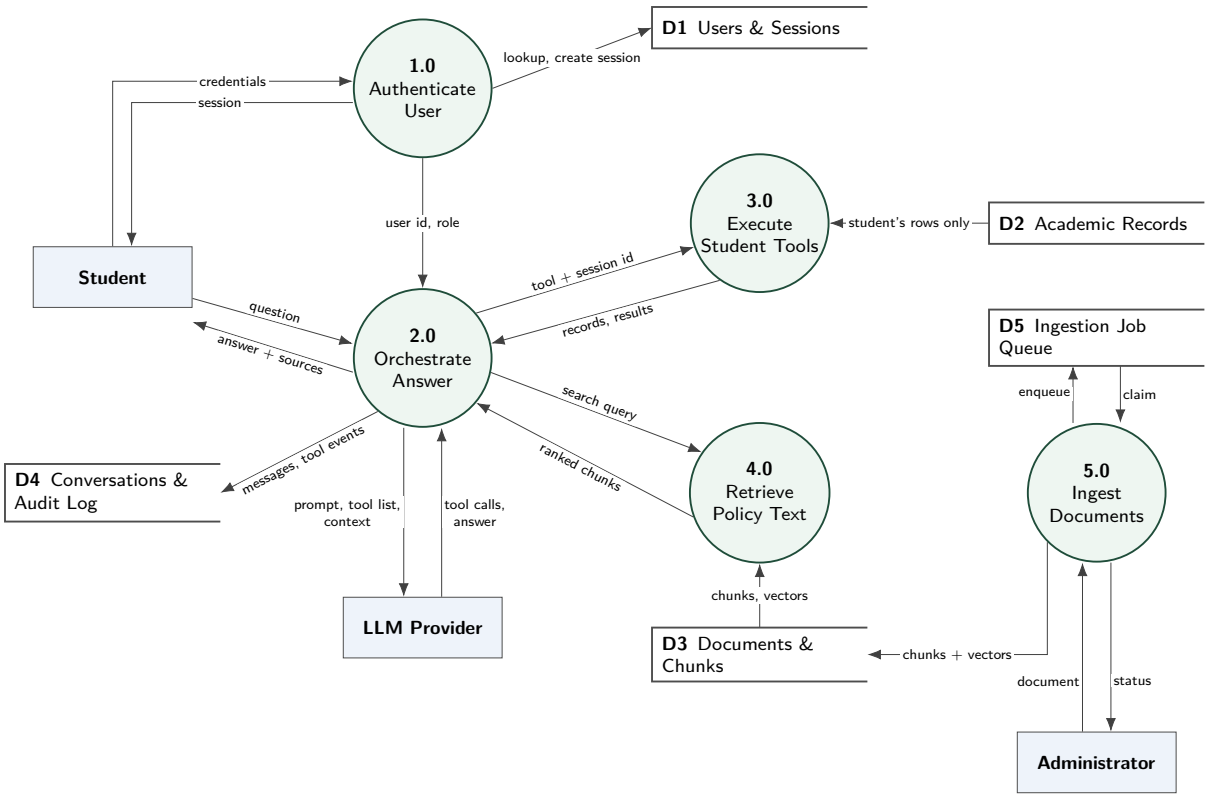


Figure B.3: Level-1 Data Flow Diagram of CampusGPT.

APPENDIX C

Issues List

Table C.1: Living register of engineering issues and roadmap resolutions

ID	Description of Engineering Issue	Status	Planned Resolution
I-01	No authorized access to real university data; all records are synthetic.	Open	Implement a university-API adapter behind the existing tool interface.
I-02	Free-tier LLM APIs impose request rate limits.	Open	Cache repeated answers; fall back to a local model.
I-03	Scanned (image-only) PDFs cannot be ingested.	Deferred	Add an OCR step to the ingestion pipeline.
I-04	Interface and answers are English only.	Deferred	Integrate Indian-language translation and speech models.
I-05	Faculty role is not implemented.	Deferred	Add faculty role with course-scoped tools.
I-06	All tools are read-only; students cannot submit requests.	Deferred	Add confirmation-gated write tools (e.g. hostel leave) with audit logging.

References

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS 33)*, 2020, pp. 9459–9474.

[3] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292–302.

[4] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[5] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd ACM SIGIR*, 2009, pp. 758–759.

[6] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[7] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS 36)*, 2023.

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[9] S. Xiao *et al.*, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. 47th ACM SIGIR*, 2024, pp. 641–649.

[10] K. Greshake *et al.*, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[11] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: `https://genai.owasp.org/llm-top-10/`

[12] Government of India, *The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023)*, New Delhi, 2023.

[13] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, Art. 248, 2023.

[14] C. Larman and V. R. Basili, "Iterative and incremental development: A brief history," *Computer*, vol. 36, no. 6, pp. 47–56, 2003.

[15] OWASP Foundation, "OWASP Top 10:2021 Web Application Security Risks," 2021. [Online]. Available: `https://owasp.org/Top10/`

[16] ISO/IEC, *Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model*, ISO/IEC 25010:2023, 2023.

[17] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the ACL: System Demonstrations*, 2024, pp. 150–158.

[18] S. Ramírez, "FastAPI documentation," 2024. [Online]. Available: `https://fastapi.tiangolo.com/`

[19] Vercel, "Next.js documentation," 2024. [Online]. Available: `https://nextjs.org/docs`

[20] The PostgreSQL Global Development Group, "PostgreSQL documentation," 2024. [Online]. Available: `https://www.postgresql.org/docs/`

[21] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2024. [Online]. Available: `https://github.com/pgvector/pgvector`